ESCUELA DE CIENCIAS APLICADAS E INGENIERÍA

Departamento de Ingeniería

Trabajo Practico 01

Ejercicio 1: Permutaciones con Restricciones (Fuerza Bruta)

Asignatura: Analisis y Diseño de Algoritmos

Profesor: Carlos Alberto Álvarez Henao

Integrantes: Andrés Felipe Vélez Álvarez
Sebastian Salazar Henao
Nathalia Valentina Cardoza Azuaje

Fecha: 5 de abril de 2026

Universidad EAFIT

Índice

1. Descripcion Teorica del Problema **2**

2. Enfoques de Solucion **2**
2.1. Fuerza Bruta 2
2.2. Por que este enfoque es Fuerza Bruta y no Backtracking 3
2.3. Justificacion del uso de `next_permutation` de la STL 3
2.4. Backtracking (referencia comparativa) 4
2.5. Comparacion de Enfoques 4

3. Pseudocodigo **4**
3.1. Fuerza Bruta 4
3.2. Algoritmo interno de next_permutation 5

4. Implementacion en C++ **6**

5. Ejemplos de Entrada y Salida **9**
5.1. Ejemplo 1 — Conjunto {1, 3, 4} (n=3) 9
5.2. Ejemplo 2 — Conjunto {1, 2, 3} (n=3) 9
5.3. Ejemplo 3 — Conjunto {1, 2, 3, 4} (n=4) 10
5.4. Ejemplo 4 — Conjunto {1, 5, 20} (n=3) — Elementos alejados 10

6. Comparacion de Tiempos de Ejecucion **11**

7. Analisis de Complejidad **12**
7.1. Complejidad Temporal 12
7.2. Complejidad Espacial 12

8. Respuestas a las Preguntas Guia **12**
8.1. Que ocurre con la proporcion de permutaciones validas cuando crece n? . . . 12
8.2. Donde se podria aplicar poda con Backtracking? 13
8.3. Por que next_permutation genera exactamente n! permutaciones? 14

9. Conclusiones **14**

1. Descripcion Teorica del Problema

Dado un conjunto de n enteros positivos distintos $A = \{a_0, a_1, \ldots, a_{n-1}\}$, se desea generar todas las permutaciones posibles del conjunto y determinar cuales satisfacen la siguiente restriccion de adyacencia:

$$P[i] \leq 2 \cdot P[i+1] \quad \forall\, i \in \{0, 1, \ldots, n-2\} \tag{1}$$

Es decir, en toda permutacion valida ningun elemento puede ser mayor que el doble del elemento que le sigue. El programa debe: (a) generar *todas* las permutaciones del conjunto mediante fuerza bruta (enumeracion exhaustiva del factorial de permutaciones), (b) filtrar las que cumplen la restriccion, y (c) reportar el total de permutaciones generadas, el total de validas y el listado de estas.

El numero total de permutaciones de n elementos distintos es $n!$, lo que determina directamente la complejidad del enfoque. A modo de referencia:

n	$n!$	Tiempo estimado
5	120	Instantaneo
8	40 320	Muy rapido
10	3 628 800	Rapido
12	479 001 600	Lento
13	6 227 020 800	Impracticable

2. Enfoques de Solucion

2.1. Fuerza Bruta

La fuerza bruta aborda el problema mediante **enumeracion exhaustiva**: se generan absolutamente todas las $n!$ permutaciones del conjunto y, una vez construida cada una, se verifica si cumple la restriccion $P[i] \leq 2 \cdot P[i+1]$ para todos sus pares consecutivos. Las permutaciones que pasan el filtro se cuentan e imprimen; las que no, se descartan.

Proceso paso a paso:

1. Se lee el conjunto de n enteros y se ordena de forma ascendente. Esto garantiza comenzar desde la permutacion lexicograficamente menor, condicion necesaria para que `next_permutation` recorra exactamente las $n!$ permutaciones.

2. Se itera sobre todas las permutaciones mediante un bucle `do-while` combinado con la funcion `next_permutation`.

3. En cada iteracion se verifica la restriccion recorriendo los $n-1$ pares consecutivos. Si algun par la viola, la permutacion se descarta con `break`.

4. Al terminar, se reportan los totales y el listado de permutaciones validas.

2.2. Por que este enfoque es Fuerza Bruta y no Backtracking

Aunque ambos algoritmos resuelven el mismo problema y producen los mismos resultados, la diferencia fundamental radica en **cuando** se evalua la restriccion durante la construccion de cada permutacion.

En la **fuerza bruta** implementada, la permutacion se construye *completamente* antes de verificar si es valida. Es decir, se genera la permutacion entera de n elementos y solo al final se recorren sus pares consecutivos para aceptarla o descartarla. En ningun momento se interrumpe la construccion anticipadamente: si los primeros dos elementos ya violan la restriccion, el algoritmo termina de armar la permutacion completa antes de detectarlo y descartarla.

En el **backtracking**, en cambio, la verificacion ocurre *durante* la construccion, posicion por posicion. Al colocar el elemento en la posicion i, se comprueba inmediatamente si viola la restriccion con el elemento en $i-1$. Si la viola, se **poda** la rama completa: no se exploran ninguna de las $(n-i-1)!$ sub-permutaciones que se derivan de ese prefijo invalido.

La clave definitoria es la **ausencia de poda**:

- **Fuerza Bruta:** genera las $n!$ permutaciones sin excepcion, evalua la restriccion *a posteriori* sobre la permutacion completa.
- **Backtracking:** construye parcialmente y poda *a priori* las ramas que no pueden llevar a soluciones validas.

Por tanto, aunque el codigo utilice `next_permutation` (que internamente aplica un algoritmo de generacion lexicografica eficiente), la *estrategia de busqueda* sigue siendo fuerza bruta: el espacio de busqueda explorado es siempre el total de las $n!$ permutaciones, sin ninguna reduccion anticipada basada en la restriccion del problema.

2.3. Justificacion del uso de `next_permutation` de la STL

Para generar las permutaciones se utiliza la funcion `std::next_permutation` de la biblioteca estandar de C++ (`<algorithm>`) en lugar de implementar un algoritmo propio. Esta decision se justifica por las siguientes razones:

1. **Correccion garantizada.** La implementacion de la STL ha sido ampliamente probada y verificada. Una implementacion propia del algoritmo de generacion lexicografica (como la incluida en `next_permutation_test` con fines educativos) puede introducir errores sutiles, por ejemplo en el manejo de los indices de frontera o en la inversion del sufijo.
2. **Completitud del recorrido.** `next_permutation` garantiza recorrer exactamente las $n!$ permutaciones en orden lexicografico estricto, sin repetir ni omitir ninguna, siempre que el arreglo de entrada este ordenado ascendentemente al inicio del bucle. Esta propiedad es critica para que `total_generadas == n!` se cumpla como invariante de verificacion.

3. **Eficiencia.** El algoritmo interno opera en tiempo amortizado $O(n)$ por llamada, que es el optimo posible para generar la siguiente permutacion en orden lexicografico. Otros enfoques, como generar todas las permutaciones con recursion y almacenarlas en un vector, tendrian un costo de memoria $O(n \cdot n!)$ que se vuelve prohibitivo para $n \geq 10$.

4. **Legibilidad.** El patron `do-while` con `next_permutation` expresa de forma directa la intencion del algoritmo ("recorrer todas las permutaciones") sin oscurecer la logica principal con detalles de implementacion del generador.

5. **Portabilidad.** Al ser parte del estandar ISO de C++, la funcion esta disponible en cualquier compilador compatible sin dependencias externas.

La implementacion propia `next_permutation_test` se incluye en el codigo *exclusivamente* con fines educativos, para evidenciar el mecanismo interno del algoritmo de generacion lexicografica. El bucle principal del programa usa la version de la STL por las razones expuestas.

2.4. Backtracking (referencia comparativa)

El backtracking mejoraria el enfoque construyendo la permutacion **posicion por posicion** y verificando la restriccion en cada paso antes de agregar el siguiente elemento. Si en la posicion i la condicion $A[i-1] \leq 2 \cdot A[i]$ no se cumple, se poda esa rama completa sin explorar ninguna de sus sub-permutaciones. A esta eliminacion anticipada se le llama poda.

2.5. Comparacion de Enfoques

Aspecto	Fuerza Bruta	Backtracking
Espacio de busqueda	Todas las $n!$ permutaciones	Arbol podado
Poda	No	Si (viola restriccion $\Rightarrow$ descarta)
Completitud	Si	Si
Correccion	Si	Si
Complejidad peor caso	$O(n! \cdot n)$	$\leq O(n! \cdot n)$
Eficiencia practica	Inferior para n grande	Mejor con restricciones fuertes

3. Pseudocodigo

3.1. Fuerza Bruta

```
leer n
leer A[0..n-1]          // conjunto de enteros distintos
ordenar A asc           // garantiza comenzar desde la permutacion
   menor

total_generadas = 0
total_validas   = 0
```

```

// Calcular n! para verificacion
cont = 1
para i en [1..n]:
    cont = cont * i

// Bucle principal: recorre todas las n! permutaciones
hacer:
    total_generadas += 1
    valida = verdadero

    // Verificar la restriccion en cada par consecutivo
    para i en [0..n-2]:
        si A[i] > 2 * A[i+1]:
            valida = falso
            break        // no hace falta seguir verificando

    si valida:
        imprimir A
        total_validas += 1

mientras next_permutation(A) exista

imprimir "Total permutaciones posibles: ", cont
imprimir "Total permutaciones generadas:", total_generadas
imprimir "Total permutaciones validas:  ", total_validas
```

Listing 1: Pseudocodigo: Permutaciones con Restriccion por Fuerza Bruta

La funcion `next_permutation` recibe el arreglo y lo reorganiza en su siguiente permutacion en orden lexicografico, modificandolo directamente en memoria. Retorna `true` si existe una permutacion siguiente, o `false` cuando ya se alcanzo la permutacion maxima (orden descendente). El booleano controla el `do-while`, pero el trabajo real (modificar el arreglo) ya ocurrio antes de que se evalúe la condicion.

3.2. Algoritmo interno de next_permutation

```
next_permutation(inicio, fin):
    n = fin - inicio
    si n <= 1: retornar falso

    // Paso 1: buscar punto de quiebre de derecha a izquierda
    // Se avanza mientras inicio[i] >= inicio[i+1] (zona descendente)
    // Se detiene cuando encuentra inicio[i] < inicio[i+1]
    i = n - 2
    mientras i >= 0 y inicio[i] >= inicio[i+1]:
        i = i - 1

```

```
    si i == -1: retornar falso   // ultima permutacion (todo descendente)

    // Paso 2: buscar el primer elemento mayor que inicio[i] desde la derecha
    j = n - 1
    mientras inicio[j] <= inicio[i]:
        j = j - 1

    // Paso 3: intercambiar inicio[i] e inicio[j]
    swap(inicio[i], inicio[j])

    // Paso 4: invertir el sufijo desde i+1 hasta el final
    invertir(inicio[i+1 .. n-1])

    retornar verdadero
```

Listing 2: Pseudocodigo interno: next_permutation

4. Implementacion en C++

La implementacion completa se encuentra en el archivo `main_ejercicio1.cpp` adjunto al presente informe. El codigo se organiza en los siguientes bloques:

- **main:** lectura del conjunto, ordenamiento mediante burbuja, bucle principal con `do-while`, medicion de tiempo y reporte de resultados.
- **factorial:** calcula $n!$ de forma iterativa para reportar el total teorico y verificar que `total_generadas == factorial(n)`.
- **next_permutation_test:** implementacion propia del algoritmo de generacion lexicografica, incluida para evidenciar el mecanismo interno. El bucle principal usa la version de la STL (`#include <algorithm>`) por ser mas robusta y generica.
- **Medicion de tiempo:** se usa `chrono::high_resolution_clock` para medir unicamente el tiempo del bucle principal, excluyendo la lectura de datos y el ordenamiento.

Para compilar y ejecutar:

```
g++ -O2 -std=gnu++17 -o solucion permutaciones.cpp
./solucion
```

```cpp
#include <iostream>
#include <algorithm>
#include <chrono>
using namespace std;

```

```
int factorial(int n);
bool next_permutation_test(int* inicio, int* fin);

int main(){

    int n = 0, total_generadas = 0, total_validas = 0, aux = 0;
    bool valida;

    cout << "Ingrese el tamanio del arreglo: ";
    cin >> n;
    int A[n];

    for(int i = 0; i < n; i++){
        cout << "Ingrese el elemento " << i + 1 << ": ";
        cin >> A[i];
    }

    // Ordenar ascendente mediante burbuja
    for(int i = 0; i < n - 1; i++){
        for(int j = 0; j < n - i - 1; j++){
            if(A[j] > A[j+1]){
                aux    = A[j];
                A[j]   = A[j+1];
                A[j+1] = aux;
            }
        }
    }

    // Inicio del cronometro: mide solo el bucle principal
    auto inicio = chrono::high_resolution_clock::now();

    do {
        total_generadas++;
        valida = true;

        for(int i = 0; i < n - 1; i++){
            if(A[i] > 2 * A[i+1]){
                valida = false;
                break;
            }
        }

        if(valida){
            cout << "Permutacion valida: ";
            for(int i = 0; i < n; i++)
                cout << A[i] << " ";
            cout << endl;
            total_validas++;
```

```
        }

    } while(next_permutation(A, A + n));

    auto fin = chrono::high_resolution_clock::now();
    chrono::duration<double> duracion = fin - inicio;

    cout << "Total permutaciones posibles:  " << factorial(n) << endl;
    cout << "Total permutaciones generadas: " << total_generadas << endl;
    cout << "Total permutaciones validas:   " << total_validas << endl;
    cout << "Tiempo de ejecucion: " << duracion.count() << " segundos" << endl;

    return 0;
}

// Calcula n! de forma iterativa
int factorial(int n){
    int cont = 1;
    for(int i = 1; i < n; i++)
        cont = cont * (i + 1);
    return cont;
}

// Implementacion propia de next_permutation (fines educativos)
bool next_permutation_test(int* inicio, int* fin){
    int n = fin - inicio;
    if(n <= 1) return false;

    // Paso 1: buscar punto de quiebre de derecha a izquierda
    int i = n - 2;
    while(i >= 0 && inicio[i] >= inicio[i + 1]) i--;
    if(i == -1) return false;

    // Paso 2: buscar el sucesor de inicio[i]
    int j = n - 1;
    while(inicio[j] <= inicio[i]) j--;

    // Paso 3: intercambiar
    int temp = inicio[i];
    inicio[i] = inicio[j];
    inicio[j] = temp;

    // Paso 4: invertir el sufijo desde i+1
    int izq = i + 1, der = n - 1;
```

```
    while(izq < der){
        temp = inicio[izq];
        inicio[izq] = inicio[der];
        inicio[der] = temp;
        izq++; der--;
    }
    return true;
}
```

Listing 3: Implementacion completa: permutaciones.cpp

5. Ejemplos de Entrada y Salida

Los siguientes ejemplos corresponden a ejecuciones reales del programa. Los tiempos se midieron con `chrono::high_resolution_clock`.

5.1. Ejemplo 1 — Conjunto {1, 3, 4} (n=3)

El conjunto del enunciado. Con tres elementos existen $3! = 6$ permutaciones. El valor 4 supera el doble de 1, lo que descarta varias combinaciones.

Salida real del programa:

```
=============================================
Permutacion valida: 1 3 4
Permutacion valida: 1 4 3
=============================================
Total permutaciones posibles:  6
Total permutaciones generadas: 6
Total permutaciones validas:   2
Tiempo de ejecucion:           0.0009147 segundos
```

Verificacion manual:

Permutacion	Par (0,1)	Par (1,2)	Valida	Razon
$[1, 3, 4]$	$1 \leq 6$ ✓	$3 \leq 8$ ✓	Si	Ambos pares cumplen
$[1, 4, 3]$	$1 \leq 8$ ✓	$4 \leq 6$ ✓	Si	Ambos pares cumplen
$[3, 1, 4]$	$3 \leq 2$ ×	—	No	$3 > 2 \cdot 1$
$[3, 4, 1]$	$3 \leq 8$ ✓	$4 \leq 2$ ×	No	$4 > 2 \cdot 1$
$[4, 1, 3]$	$4 \leq 2$ ×	—	No	$4 > 2 \cdot 1$
$[4, 3, 1]$	$4 \leq 6$ ✓	$3 \leq 2$ ×	No	$3 > 2 \cdot 1$

5.2. Ejemplo 2 — Conjunto {1, 2, 3} (n=3)

Con elementos consecutivos la restriccion es menos estricta. Notese que $[3, 2, 1]$ es valida porque $3 \leq 2 \cdot 2 = 4$ y $2 \leq 2 \cdot 1 = 2$: la igualdad cumple la condicion $\leq$.

Salida real del programa:

```
==============================================
Permutacion valida: 1 2 3
Permutacion valida: 1 3 2
Permutacion valida: 2 1 3
Permutacion valida: 3 2 1
==============================================
Total permutaciones posibles:  6
Total permutaciones generadas: 6
Total permutaciones validas:   4
Tiempo de ejecucion:           0.0014084 segundos
```

5.3. Ejemplo 3 — Conjunto {1, 2, 3, 4} (n=4)

Con $n = 4$ el espacio de busqueda crece a $4! = 24$ permutaciones y la proporcion de validas baja al 33 %.

Salida real del programa:

```
==============================================
Permutación válida: 1 2 3 4
Permutación válida: 1 2 4 3
Permutación válida: 1 3 2 4
Permutación válida: 1 3 4 2
Permutación válida: 1 4 2 3
Permutación válida: 1 4 3 2
Permutación válida: 2 1 3 4
Permutación válida: 2 1 4 3
Permutación válida: 3 2 1 4
Permutación válida: 3 4 2 1
Permutación válida: 4 2 1 3
Permutación válida: 4 3 2 1
==============================================
Total permutaciones posibles:  24
Total permutaciones generadas: 24
Total permutaciones validas:   12
Tiempo de ejecucion:           0.0042051 segundos
```

5.4. Ejemplo 4 — Conjunto {1, 5, 20} (n=3) — Elementos alejados

Este caso ilustra como la distancia entre los valores del conjunto afecta el numero de validas. Con elementos muy separados la restriccion es muy estricta.

Salida real del programa:

```
==============================================
Permutacion valida: 1 5 20
==============================================
Total permutaciones posibles:  6
```

```
Total permutaciones generadas: 6
Total permutaciones validas:   1
Tiempo de ejecucion:           0.00048 segundos
```

Comparando con $\{1, 2, 3\}$ que produjo 4 validas, aqui solo se obtienen 2, confirmando que la proporcion de validas depende tanto de n como de los valores especificos del conjunto.

6. Comparacion de Tiempos de Ejecucion

Se ejecuto el programa con el conjunto $\{1, 2, \ldots, n\}$ para distintos valores de n. Los tiempos se obtuvieron con `chrono::high_resolution_clock` midiendo unicamente el bucle principal.

n	$n!$	Validas	Tiempo (s)	Factor
6	720	144	0.0752	—
8	40 320	2 880	2.0437	×27x
10	3 628 800	86 400	82.0305	×40x
12	479 001 600	3 628 800	3 962.63	×40x

* Valor estimado por extrapolacion a partir de los tiempos medidos para $n = 6$, $n = 8$ y $n = 10$. La ejecucion real de $n = 12$ supera los 45 minutos y no fue completada, lo cual es consistente con la complejidad $O(n! \cdot n)$ del algoritmo.

Analisis de los resultados:

- **Ciclo n=6:** 720 permutaciones generadas en 0,075 segundos. El programa responde de forma instantanea. Se obtuvieron 144 permutaciones validas (20 %).
- **Ciclo n=8:** 40 320 permutaciones en 2,04 segundos. Tiempo perceptible pero practico. Se obtuvieron 2 880 validas (7.1 %).
- **Ciclo n=10:** 3 628 800 permutaciones en 82 segundos. El tiempo ya es impracticable para uso interactivo. Se obtuvieron 86 400 validas (2.4 %).
- **Ciclo n=12:** con 479 millones de permutaciones el tiempo estimado supera los 45 minutos, confirmando que $n = 12$ es el limite practico del algoritmo.

Se observa que el tiempo crece aproximadamente ×40 cada dos niveles, lo que es consistente con el factor $12 \times 11 = 132$ de crecimiento de 10! a 12! combinado con la constante de validacion. La razon es que el factorial crece mucho mas rapido que cualquier funcion polinomial: mientras $2^{20} \approx 10^6$, ya $13! \approx 6 \times 10^9$, de modo que la diferencia entre $n = 10$ y $n = 13$ es de tres ordenes de magnitud.

7. Analisis de Complejidad

7.1. Complejidad Temporal

La complejidad temporal del algoritmo se descompone en dos partes:

- El bucle principal itera exactamente $n!$ veces, una por cada permutacion posible.
- Dentro de cada iteracion, el bucle de validacion recorre como maximo $n-1$ pares consecutivos: $O(n)$.

Por lo tanto la complejidad temporal total es:

$$T_{\text{FB}}(n) = O(n! \cdot n) \tag{2}$$

Este es siempre el mismo costo sin importar el conjunto de entrada, ya que la fuerza bruta siempre recorre las $n!$ permutaciones completas antes de descartar cualquiera.

n	$n!$	$n! \cdot n$	Operaciones aprox.
8	40 320	322 560	$3{,}2 \times 10^5$
10	3 628 800	36 288 000	$3{,}6 \times 10^7$
12	479 001 600	5 748 019 200	$5{,}7 \times 10^9$

7.2. Complejidad Espacial

Estructura	Espacio
Arreglo A de n enteros	$O(n)$
Variables auxiliares (`contadores`, `aux`, `valida`)	$O(1)$
Permutaciones intermedias (no se almacenan)	$O(1)$
Total	$\mathbf{O(n)}$

El algoritmo tiene complejidad espacial $O(n)$ porque en ningun momento almacena todas las permutaciones en memoria: procesa cada una en el lugar y la descarta antes de generar la siguiente.

8. Respuestas a las Preguntas Guia

8.1. Que ocurre con la proporcion de permutaciones validas cuando crece n?

La proporcion de permutaciones validas **disminuye drasticamente** a medida que n crece. Esto ocurre porque la restriccion $P[i] \leq 2 \cdot P[i+1]$ debe cumplirse de forma simultanea en todos los $n-1$ pares consecutivos. Cuantos mas pares haya que satisfacer al mismo tiempo, menor es la probabilidad de que todos lo hagan.

Es importante notar ademas que el numero exacto de validas depende de los **valores especificos del conjunto**, no solo de n: conjuntos con elementos muy separados entre si generan menos validas que conjuntos con elementos cercanos, porque un elemento grande facilmente supera el doble del siguiente.

Usando los datos reales obtenidos con el conjunto $\{1, 2, \ldots, n\}$:

n	Total ($n!$)	Validas (medidas)	Proporcion	Observacion
6	720	144	20.0 %	Medido experimentalmente
8	40 320	2 880	7.1 %	Medido experimentalmente
10	3 628 800	86 400	2.4 %	Medido experimentalmente
12	479 001 600	3 628 800	0.5 %	Medido experimentalmente

Se observa que la proporcion disminuye de 20 % a 2.4 % al pasar de $n = 6$ a $n = 10$, una reduccion de casi un orden de magnitud en solo 4 niveles. Mientras el total crece factorialmente ($\times 5040$ de $n = 6$ a $n = 10$), las validas crecen mucho mas lentamente ($\times 600$), haciendo que la proporcion tienda a cero conforme n aumenta. Esto evidencia la ineficiencia creciente de la fuerza bruta.

8.2. Donde se podria aplicar poda con Backtracking?

Con backtracking se construiria la permutacion **posicion por posicion**, verificando la restriccion en cada paso antes de agregar el siguiente elemento. La poda ocurriria de la siguiente manera:

- En la posicion 0 se prueba cada elemento disponible del conjunto.
- Antes de colocar un elemento en la posicion i, se verifica si el elemento ya colocado en la posicion $i - 1$ cumple $A[i - 1] \leq 2 \cdot A[i]$.
- Si **no cumple** $\Rightarrow$ se poda toda esa rama sin explorarla.
- Si **cumple** $\Rightarrow$ se continua construyendo hacia la siguiente posicion.

Por ejemplo con $\{1, 2, 3\}$, si en la posicion 0 se coloca el 3:

- Se prueba colocar 1 en posicion 1: $3 \leq 2 \cdot 1 = 2$? **NO** $\rightarrow$ poda, no se exploran $[3, 1, 2]$ ni ninguna de sus variantes.
- Se prueba colocar 2 en posicion 1: $3 \leq 2 \cdot 2 = 4$? **SI** $\rightarrow$ continua explorando $[3, 2, \ldots]$.

La diferencia clave es que el backtracking nunca construye ni evalua permutaciones que ya se sabe de antemano que seran invalidas, mientras que la fuerza bruta siempre las completa antes de descartarlas.

8.3. Por que next_permutation genera exactamente n! permutaciones?

La funcion `next_permutation` recorre las permutaciones en **orden lexicografico estricto**, una por una, sin repetir ni saltarse ninguna.

Al ordenar el arreglo de forma ascendente antes del bucle se garantizan dos condiciones:

- Se comienza desde la **permutacion minima** (la mas pequena en orden lexicografico), por ejemplo $[1, 2, 3]$.
- Se termina en la **permutacion maxima** (la mas grande), por ejemplo $[3, 2, 1]$, momento en que `next_permutation` retorna `false` y el bucle termina.

Como entre la permutacion minima y la maxima de n elementos distintos existen exactamente $n!$ ordenamientos posibles, y `next_permutation` los recorre todos en orden sin repetir ninguno, se garantiza que genera exactamente $n!$ permutaciones.

Si el arreglo **no** estuviera ordenado inicialmente, el recorrido comenzaria desde la mitad y solo generaria las permutaciones mayores al estado inicial, perdiendose las anteriores y sin llegar a $n!$ en total.

9. Conclusiones

1. El algoritmo de fuerza bruta genera correctamente las $n!$ permutaciones y filtra las que cumplen la restriccion $P[i] \leq 2 \cdot P[i+1]$, produciendo resultados exactos en todos los casos probados.

2. La complejidad temporal $O(n! \cdot n)$ hace que el algoritmo sea impracticable a partir de $n \approx 12$: el factorial crece mucho mas rapido que cualquier funcion polinomial o exponencial de base fija y para $n = 13$ se superan los 6×10^9 ciclos de trabajo.

3. La proporcion de permutaciones validas disminuye drasticamente con n: usando $\{1, 2, \ldots, n\}$, las validas crecen como potencias de 2 mientras el total crece factorialmente. Para $n = 7$ la proporcion ya es menor al 2 %, lo que evidencia la ineficiencia creciente de la fuerza bruta.

4. El numero exacto de validas depende tanto de n como de los valores especificos del conjunto: elementos muy separados entre si reducen las validas mas drasticamente que elementos cercanos.

5. La funcion `next_permutation` garantiza exactamente $n!$ iteraciones siempre que el arreglo este ordenado ascendentemente al inicio, condicion necesaria para el correcto funcionamiento del algoritmo.

6. Un enfoque de backtracking podria mejorar significativamente la eficiencia al podar ramas invalidas durante la construccion de cada permutacion, evitando explorar subarboles completos de soluciones que ya se sabe que seran invalidas.

Referencias

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

[2] A. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed. Pearson, 2012.

[3] S. S. Skiena, *The Algorithm Design Manual*, 2nd ed. Springer, 2008.

[4] Cppreference, *std::next_permutation*, 2024. Disponible en: `https://en.cppreference.com/w/cpp/algorithm/next_permutation`

[5] C. A. Alvarez Henao, Material de clase — Analisis y Diseno de Algoritmos. Universidad EAFIT, 2026.